第5章 第8节 文档分块：Chunking的艺术

- 来源: https://class.imooc.com/lesson/5273#mid=104920
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: 第一个RAG应用：游戏知识问答系统
- 课节: 图文 1-2 文档分块：Chunking的艺术

1-2 文档分块：Chunking的艺术

第5章 第8节 文档分块：Chunking的艺术

> [!TIP] 阅读指南 上一节我们做了第一个RAG应用，用的是精心准备的游戏角色资料——每个文件都用【】标记好了语义块，按空行切分就能得到不错的效果。 但真实世界可不会这么友好。假设你的老板扔给你一份100页的PDF文档，说："把这个做成知识库，下周要用。" 你打开一看：扫描版、双栏排版、表格跨页、页眉页脚到处飞…… 这就是本节要解决的问题——如何把长文档切成合适的小块（chunk），让RAG检索得更准、回答得更好。

8.1 为什么分块这么重要

某法律咨询公司做了一个AI法律顾问，把所有法律条文都灌进向量库。看起来似乎很完美：1000份法律文档、50万字法律条文，用户问"合同违约怎么赔偿？"。

结果却很糟糕——AI检索到的是这样一段文本：

> 第三章 合同法 第一节 合同的订立 合同是平等主体的自然人、法人、其他组织之间设立、变更、终止民事权利义务关系的协议。婚姻、收养、监护等有关身份 关系的协议，适用其他法律的规定。第二节 合同的效力 合同生效后，当事人应当按照约定全面履行自己的义务。第三节 合同的履行当事人一方不履行合同义务或者履行合同义务不符合 约定的，应当承担继续履行、采取补救措施或者赔偿损失等违约 责任。

检索结果包含了"违约"和"赔偿"的段落，但把整个章节（3000字）都塞进去了。关键的"赔偿标准"被淹没在大量无关内容中，AI无法准确定位核心信息。

如果换一种切分方式，把每个条款单独切成一块：

> 【chunk_42】 第三节 合同的履行 当事人一方不履行合同义务或者履行合同义务不符合约定的，应当承担继续履行、采取补救措施或者赔偿损失等违约责任。 【chunk_43】 违约赔偿标准： 约定违约金的，按约定计算 未约定的，按实际损失计算 损失难以确定的，参照同类合同标准

现在检索到的是chunk_43，信息精准、答案准确。这就是分块的好处。

好的chunk需要同时满足三个条件：

- 最小 — 精准（不包含无关信息、Token消耗低、检索更准确）

最小

- 完整 — 可独立理解（不会出现指代不明，有足够上下文）

完整

- 语义单元 — 完整概念（一个chunk表达一个完整的意思，不在句子或段落逻辑中间切断）

8.2 四个核心挑战

语义边界模糊不清

看这段文本（150字）：

```
胡桃是璃月的堂主，性格古灵精怪。她擅长使用长柄武器，
元素类型是火。作为火系输出角色，胡桃的核心机制是血量
管理—E技能会消耗30%生命值，但会大幅提升攻击力。
因此，胡桃最适合的圣遗物是生命值加成套装。推荐使用
魔女4件套，主词条选择生命值沙漏、火伤杯、暴击头。
```

以下三种方案各有利弊：

- 方案A：全部算一个chunk（150字）——信息完整，但用户问"圣遗物推荐"时会检索到人物介绍，有噪音
- 方案B：切成3个chunk（人物介绍 + 核心机制 + 圣遗物推荐）——检索精准，但指代关系丢失，语义不完整
- 方案C：切成2个chunk（人物+机制 | 配装推荐）——平衡精准度和完整性，但边界模糊，机制和配装有强依赖关系

没有完美答案，只能在精准和完整性之间取舍。

上下文依赖无处不在

看这段技能描述：

```
【chunk_1】E技能：蝶引来生
消耗当前生命值的30%，进入彼岸蝶舞状态。

【chunk_2】彼岸蝶舞状态下：
- 攻击力提升：基于生命值上限
- 攻击附带火元素伤害
- 持续时间9秒

【chunk_3】特殊机制：
生命值低于50%时，火伤加成额外提升33%

【chunk_4】配队推荐：
需要护盾角色（钟离/迪奥娜）保护
```

单独看每个chunk都有问题：

- chunk_2 不知道"彼岸蝶舞"是什么
- chunk_3 不知道是哪个技能的机制
- chunk_4 不知道为什么需要护盾

往前合并？chunk_2 要包含 chunk_1 的信息。往后也得合并？chunk_2 还要包含 chunk_3 的机制说明。全部合并？那和不切分没区别。

矛盾所在：往前合并，往后也得合并；往后合并，往前又不够用。

矛盾所在

不同视角不同需求

同一段文本，不同用户关注点不同：

```
胡桃的核心输出手法是：开E → 重击 × 9 → Q爆发。
这套循环需要配合行秋/夜兰提供水元素附着，触发蒸发反应。
理论伤害上限很高，但需要精准控制血量和取消后摇。
```

- 用户A（新手）问"胡桃怎么打输出？" → 关心技能循环，希望检索到开E → 重击 → Q

- 用户B（配队研究）问"胡桃配什么辅助？" → 关心配队建议，希望检索到行秋/夜兰
- 用户C（伤害计算）问"胡桃输出上限多少？" → 关心理论伤害，希望检索到伤害上限、蒸发反应

一段文本只有一个切分方案，不可能满足所有用户。

既然人工规则面临这么多难题，一个自然的想法是：让大模型来自动识别语义边界。

自动识别语义边界的成本

大模型确实能识别语义边界，但工程中面临成本、时间、稳定性三个问题。

成本问题

假设有1000份文档、每份5000字，用大模型识别最佳切分点：

```
提示词：
请分析以下文本，识别语义边界，给出最佳切分方案。
要求：保持每个chunk的语义完整性和独立性。

文本：[5000字的文档]
```

- 每份文档需要调用1次LLM（5000 tokens输入 + 500 tokens输出）
- 1000份文档 = 5,500,000 tokens
- 按通义千问Plus计费（¥0.02/1K tokens）≈ ¥110
- 每次文档更新都要重新分析，成本持续累积

时间问题

- 每份文档分析耗时：3-10秒
- 1000份文档：50分钟 - 2.7小时
- 10万份文档需要几天甚至几周，无法实时处理

稳定性问题

大模型的输出是概率性的，同样的文档两次分析可能得到不同结果，不确定性导致难以调试和优化。

对于大规模文档处理，成本和时间不可接受。仍需要简单、高效、可控的规则方法，大模型更适合作为辅助手段。

8.3 现实的优化策略

不追求完美，追求"在当前场景下够用"。

根据文档类型选择策略

- 有明确结构（API文档、法律条文）→ 按结构切分
- 无明确结构（文章、小说）→ 递归切分
- 强调语义完整性 → 语义分块
- 追求简单快速 → 固定大小切分

测试驱动

- 准备典型问题集
- 对比不同策略的检索效果
- 选择最适合的方案

接受权衡

- 没有完美方案
- chunk大一点，信息完整但有噪音
- chunk小一点，精准但可能语义不足
- 找到平衡点

持续优化

- 分析检索失败的case
- 调整分块策略

8.4 四种分块策略详解

现在我们知道了分块的难点，接下来看看实际可用的4种策略。

固定大小分块

核心思路

简单粗暴，按固定字符数或Token数切分。

实现原理

> 文本长度：10000字 Chunk Size：500字 Overlap：50字（重叠部分，避免截断语义） 切分结果： chunk_1: 文本[0:500] chunk_2: 文本[450:950] # 从450开始，与chunk_1重叠50字 chunk_3: 文本[900:1400] ...

优势：实现简单、速度快、可控性强（chunk大小固定）。

优势

劣势：可能在句子中间切断，完全无视文档结构，语义完整性差。

劣势

适用场景：文档结构不重要、追求极致速度、原型验证阶段。

适用场景

递归分块

优先用大分隔符切，切不下再用小分隔符，层层递归。

```
分隔符优先级：
1. 章节分隔符（\n\n\n）    - 最粗粒度
2. 段落分隔符（\n\n）      - 中粒度
3. 句子分隔符（\n）        - 细粒度
4. 句号（。）              - 更细粒度
5. 逗号（，）              - 最细粒度

递归逻辑：
- 用分隔符1切分，如果某部分还是太大
  → 用分隔符2再切这部分
  → 如果还是太大，用分隔符3
  → ...
  → 直到满足大小要求
```

示例

假设有这段文本（摘自《红楼梦》第十七回），限制100字/块：

第一章 大观园潇湘馆

于是出亭过池，一山一石，一花一木，莫不着意观览。忽抬头见前面一带粉垣，里面数楹修舍，有千百竿翠竹遮映。众人都道："好个所在！"（约60字）

于是大家进入，只见入门便是曲折游廊，阶下石子漫成甬路。上面小小两三间房舍，一明两暗，里面都是合着地步打就的床几椅案。从里间房内又得一小门，出去则是后院，有大株梨花兼着芭蕉。又有两间小小退步。后院墙下忽开一隙，得泉一派，开沟仅尺许，灌入墙内，绕阶缘屋至前院，盘旋竹下而出。（约140字）

第二章 怡红院

一入门，两边都是游廊相接。院中点衬几块山石，一边种着数本芭蕉，那一边乃是一棵西府海棠，其势若伞，丝垂翠缕，葩吐丹砂。（约50字）

递归切分过程

```
第1轮：用\n\n\n切章节
  → [第一章(200字), 第二章(50字)]
  → 第二章50字 < 100字 ✓ 保留
  → 第一章200字 > 100字 ✗ 继续切分

第2轮：对第一章用\n\n切段落
  → [标题+段1(60字), 段2(140字)]
  → 段1: 60字 < 100字 ✓ 保留
  → 段2: 140字 > 100字 ✗ 继续切分

第3轮：对段2用句号切句子
  → [句1(25字), 句2(35字), 句3(30字), 句4(40字)]
  → 全部 < 100字 ✓

最终结果：
chunk_1 = "第一章 大观园潇湘馆\n\n于是出亭过池...众人都道：\"好个所在！\""(60字)
chunk_2 = "于是大家进入...阶下石子漫成甬路。"(25字)
chunk_3 = "上面小小两三间房舍...床几椅案。"(35字)
chunk_4 = "从里间房内...梨花兼着芭蕉。"(30字)
chunk_5 = "后院墙下忽开一隙...盘旋竹下而出。"(40字)
chunk_6 = "第二章 怡红院...葩吐丹砂。"(50字)
```

这个示例同时展示了直接保留（第二章、段1）和递归切分（第一章的段2）两种机制。

优势：尊重文档结构（优先按章节、段落切），保持语义相对完整，灵活适应不同文档。

劣势：可能产生大小不均匀的chunk，需要人工定义分隔符优先级，对分隔符敏感。

适用场景：有一定结构但不规范的文档，需要平衡语义和大小，90%的通用场景。

语义分块

用Embedding检测语义相似度，在话题转换处切分。

> 按句子分割文本 计算每个句子的Embedding向量 计算相邻句子的相似度 相似度低于阈值 → 话题转换 → 切分 相似度高于阈值 → 话题连贯 → 合并

核心逻辑（伪代码）

```
# 语义分块
def semantic_chunking(text, threshold=0.65):
    # 1. 按句号分句
    sentences = text.split('。')

    # 2. 调用Embedding API获取每个句子的向量
    vectors = call_embedding_api(sentences)

    # 3. 计算相邻句子的相似度，决定是否切分
    chunks = []
    current_chunk = [sentences[0]]

    for i in range(1, len(sentences)):
        # 计算余弦相似度
        similarity = cosine_similarity(vectors[i-1], vectors[i])

        if similarity < threshold:
            # 相似度低 → 话题转换 → 切分
            chunks.append(''.join(current_chunk))
            current_chunk = [sentences[i]]
        else:
            # 相似度高 → 话题连贯 → 合并
            current_chunk.append(sentences[i])

    # 添加最后一个chunk
    chunks.append(''.join(current_chunk))
    return chunks
```

- call_embedding_api() - 调用Embedding接口

```
call_embedding_api()
```

- cosine_similarity() - 计算向量相似度

```
cosine_similarity()
```

优势：最智能，能自动识别话题边界；语义完整性最好；不依赖文档结构。

劣势：需要调用Embedding API（有成本），速度慢（每个句子都要向量化），效果依赖阈值调优，大规模文档处理成本高。

文档结构分块

利用文档的原生结构（标题、章节、条目）进行切分。

常见结构类型

Markdown文档

```
# 第一章
## 3.7.3 小节
内容...

## 3.7.4 小节
内容...

# 第二章
## 3.7.5 小节
内容...
```

优势：语义完整性最好（每个chunk都是完整的条目/章节），边界清晰不会切断语义，最适合有明确结构的文档，实现简单速度快。

劣势：只适用于有结构的文档，需要针对每种文档格式定制，可能产生大小差异很大的chunk。

适用场景：Markdown文档、API文档、编程规范、法律条文等有明确结构的文档，结构化文档的首选方案。

8.5 参数调优

选择Chunk Size时可以参考这个决策树：

```
文档有明确的最小独立单元吗？
    ├─ 有（如：API的一个方法、FAQ的一个QA）
    │    → Chunk Size = 单元平均大小
    │
    └─ 没有（如：长文章、小说）
         → 问：上下文依赖强吗？
              ├─ 强 → Chunk Size大一点（1000-2000）
              └─ 弱 → Chunk Size小一点（500-1000）
```

调优方法

- 从中间值开始（500 tokens）
- 准备10个典型问题
- 测试检索准确率
- chunk太小 → 语义不完整 → 增大
- chunk太大 → 噪音太多 → 减小

Overlap在两个chunk之间保留一段共同文本，避免关键信息被切断。

看这个例子：

> 【chunk_1】胡桃的E技能会消耗30%生命值，但会大幅提升攻击力。 【chunk_2】因此，胡桃最适合的圣遗物是生命值加成套装。

用户问"为什么胡桃要堆生命值？"：

- 没有overlap：检索到chunk_2，只知道"要生命值套装"，不知道原因
- 有overlap：chunk_2的前50字包含chunk_1的后50字，检索到了"E技能消耗生命"的信息

推荐值

```
Overlap 大小 = Chunk Size 的 10%~20%

例如：
Chunk Size = 500 → Overlap = 50~100
Chunk Size = 1000 → Overlap = 100~200
```

8.6 分块策略选择

四种策略各有利弊，选择取决于文档类型和需求：

```
你的文档有明确结构（Markdown、API文档、法律条文）？
    ├─ 是 → 结构分块（最优）
    │
    └─ 否 → 对语义完整性要求极高 且 成本不敏感？
            ├─ 是 → 语义分块
            │
            └─ 否 → 递归分块（推荐）
```

8.7 下一节预告

下一节，我们用一份37页的PDF文档——《阿里巴巴Java开发手册》，来实战对比这四种策略。你将看到：固定大小切分如何把完整的规范切成两半，递归分块为何产生99个碎片，以及结构化分块如何完美处理条目化文档。

8.8 ■ 学点英语

中文	English	音标	说明
文档分块	Chunking	/ˈtʃʌŋkɪŋ/	将长文档切成语义完整的小块以便检索的预处理技术
重叠	Overlap	/ˌoʊvərˈlæp/	相邻Chunk之间共享的内容比例，通常为10%-20%
结构化分块	Structural Chunking	/ˈstrʌktʃərəl ˈtʃʌŋkɪŋ/	按标题、条款编号等文档结构元素切分
语义分块	Semantic Chunking	/sɪˈmæntɪk ˈtʃʌŋkɪŋ/	用AI判断自然段落边界进行切分
递归分块	Recursive Chunking	/rɪˈkɜːrsɪv ˈtʃʌŋkɪŋ/	先大块再细分，尝试在语义边界处切分